

Access Controls for Production Assets and Sensitive Data

Company: Cashpile
Document Date: April 26, 2026
Version: 1.0
Purpose: Response to Plaid Security Assessment Question #3

Executive Summary

Cashpile implements a defense-in-depth approach to access control, leveraging managed cloud services with built-in security controls, strict database isolation via Row Level Security (RLS), and application-level authentication and authorization. This document details the specific access controls in place to protect production assets and sensitive financial data.

1. Production Infrastructure Access

1.1 Hosting Platform

Attribute	Implementation
Primary Platform	Railway (managed Platform-as-a-Service)
Server Access	No direct SSH or console access to underlying servers
Deployment Method	Automated CI/CD via GitHub integration
Physical Security	Managed by Railway (SOC 2 Type II compliant infrastructure)

1.2 Environment Separation

Cashpile maintains strict separation between environments:

- Development:** Local development environments with sanitized test data
- Staging:** Isolated staging environment for pre-production testing
- Production:** Separate production environment with no shared credentials

1.3 Administrative Access

Control	Implementation
Dashboard Access	Railway web console with MFA-required accounts
User Roles	Role-based permissions within Railway organization
Access Logging	All dashboard actions logged by Railway
Session Management	Automatic session timeout after inactivity

2. Database Access Controls

2.1 Database Platform

Cashpile uses **Supabase** (PostgreSQL-as-a-Service) for all data storage.

2.2 Row Level Security (RLS)

All database tables containing sensitive data have Row Level Security enabled with user-specific policies:

```
-- Example: Plaid items table
CREATE POLICY "users can manage their own plaid items"
  ON public.books_plaid_items
  FOR ALL
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);

-- Example: Financial accounts table
CREATE POLICY "users can manage their own financial accounts"
  ON public.books_financial_accounts
  FOR ALL
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);

-- Example: Transactions table
CREATE POLICY "users can manage their own transactions"
  ON public.books_transactions
  FOR ALL
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);
```

2.3 Database Access Methods

Access Type	Authentication	Use Case
Client-Side	Supabase Auth JWT	User queries via application
Server-Side	Service-role key	API routes, background jobs
Direct SQL	Database password	Migrations only (restricted)

2.4 Service-Role Key Restrictions

- Service-role key bypasses RLS and is used only in server-side API routes
- Key is never exposed to client-side code or browser environments
- Key is stored as an environment variable, never committed to version control

3. Application-Level Access Controls

3.1 Authentication

Control	Implementation
Primary Method	Supabase Auth with JWT tokens
Session Duration	Configurable expiration with refresh tokens
Password Policy	Minimum 8 characters, complexity enforced
MFA	Available and enforced for administrative accounts
Session Timeout	Automatic logout after extended inactivity

3.2 Authorization

Cashpile implements multi-layer authorization:

User-Level Authorization

- Users can only access data associated with their `user_id`
- Enforced at database level via RLS policies
- Re-verified at application level for defense in depth

API Route Protection

```
// Example: All Plaid API routes require authentication
export async function POST(req: NextRequest) {
  const supabase = await createServerSupabaseClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return NextResponse.json({ error: "Unauthenticated" }, {
    status: 401 });
  // ... route logic
}
```

Cron/Internal Endpoint Protection

```
// Cron endpoints protected by secret header
const isCron = req.headers.get("x-cron-secret") ===
process.env.CRON_SECRET;
if (!isCron) return NextResponse.json({ error: "Unauthorized" }, { status:
401 });
```

3.3 Plaid-Specific Access Controls

Control	Implementation
Token Storage	Plaid access tokens encrypted at rest in database
Token Access	Never exposed to client-side code; server-side only
Token Logging	Tokens are never logged to console or logging systems

Control	Implementation
Webhook Verification	Webhook signatures verified (production environment)
API Scoping	Limited to required Plaid products only (transactions, investments)

4. Sensitive Data Handling

4.1 Data Classification

Classification	Data Types	Protection Level
Critical	Plaid access tokens, bank account credentials	Encrypted at rest, service-role only access
Sensitive	Transaction history, account balances	RLS-protected, user-scoped access
Internal	Application logs, error reports	Access restricted to development team

4.2 Encryption

Layer	Method	Status
Data at Rest	AES-256	Managed by Supabase
Data in Transit	TLS 1.3	Enforced for all connections
Database Connections	SSL/TLS	Required for all connections
API Communications	HTTPS	Enforced for all external APIs

4.3 Data Retention and Disposal

- Plaid access tokens retained only while account connection is active
- Transaction data retained per user account lifecycle
- Secure deletion via Supabase **DELETE** operations with RLS verification
- Backups retained for 30 days with encrypted storage

5. Code and Deployment Security

5.1 Version Control

Control	Implementation
Repository	Private GitHub repository
Branch Protection	Required reviews for main branch
Secret Scanning	GitHub secret scanning enabled
Access Control	Repository access limited to organization members

5.2 Secrets Management

Secret Type	Storage Method
Database credentials	Railway environment variables (encrypted)
Plaid API keys	Railway environment variables (encrypted)
Supabase keys	Railway environment variables (encrypted)
Cron secrets	Railway environment variables (encrypted)

Policy: No production secrets are ever committed to version control.

5.3 Deployment Pipeline

1. Code changes require pull request review
2. Automated tests run on pull requests
3. Deployment to production requires manual approval
4. Rollback capability available within Railway dashboard

6. Monitoring and Audit

6.1 Logging

Log Type	Retention	Access
Application logs	30 days	Development team
Database audit logs	1 year	Database administrator
Authentication events	1 year	Security administrator

6.2 Monitoring

- Error tracking via application monitoring
- Database performance monitoring via Supabase dashboard
- Infrastructure monitoring via Railway dashboard

6.3 Audit Trail

All access to sensitive data is logged with:

- Timestamp
- User identifier
- Action performed
- Resource accessed

7. Third-Party Access Controls

7.1 Vendor Security Assessment

Vendor	Service	Security Certification
Railway	Application hosting	SOC 2 Type II
Supabase	Database hosting	SOC 2 Type II
Plaid	Financial data aggregation	SOC 2 Type II, ISO 27001
GitHub	Version control	SOC 2 Type II, ISO 27001

7.2 Data Processing Agreements

Data Processing Agreements (DPAs) are in place or standard terms accepted with all vendors processing sensitive data.

8. Incident Response

8.1 Security Incident Classification

Severity	Definition	Response Time
Critical	Data breach, unauthorized access to production	1 hour
High	Token exposure, authentication bypass	4 hours
Medium	Policy violation, suspicious activity	24 hours
Low	Minor security concern	72 hours

8.2 Plaid-Specific Incident Response

In the event of a security incident involving Plaid data:

- 1. **Immediate:** Revoke affected Plaid access tokens
- 2. **Within 1 hour:** Isolate affected systems
- 3. **Within 24 hours:** Notify Plaid via security@plaid.com
- 4. **Within 72 hours:** Provide incident report with root cause analysis

9. Compliance and Certifications

9.1 Current Compliance

- GDPR compliance for EU data subjects
- CCPA compliance for California residents
- SOC 2 Type II compliance (infrastructure providers)

9.2 Planned Compliance

- SOC 2 Type II audit (planned within 12 months)

10. Contact Information

Security Inquiries: security@cashpile.app
Incident Reporting: security@cashpile.app
Data Protection Officer: security@cashpile.app

Document Control

Version	Date	Author	Changes
1.0	April 26, 2026	Cashpile Security	Initial document creation

Review Cycle: Annual
Next Review Date: April 26, 2027

This document is confidential and intended for Plaid security assessment purposes only.